WEEK 1: FIRST LLM API CALL USING OPENAI

1. Aim

To establish a connection between a Python application and a Large Language Model (LLM) using the OpenAI API and generate responses based on user prompts.

2. Software Requirements

- Python
- Visual Studio Code
- OpenAI API Key
- OpenAI Python SDK
- python-dotenv library

3. Libraries Used

1. **os:** Used to access environment variables.
2. **python-dotenv:** Used to load API keys from the .env file.
3. **openai:** Used to connect the Python application with the OpenAI API.

4. Installation Commands

Install the required libraries using the following commands:

```
pip install openai
pip install python-dotenv
```

5. Project Structure

Create the following files inside the project folder:

- main.py
- .env
- .gitignore
- requirements.txt

6. API Key Configuration

Create a `.env` file and store the API key:

`OPENAI_API_KEY=your_actual_api_key`

The API key is stored securely in the environment file instead of directly writing it in the Python program.

7. Python Program

Create a file named `main.py` and write the Python code to connect with the OpenAI API, send a prompt and display the generated response.

```
import os
from dotenv import load_dotenv
from google import genai
from google.genai import errors
# Load the API key from .env
load_dotenv()
api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise RuntimeError("GEMINI_API_KEY was not found in your .env file.")
client = genai.Client(api_key=api_key)
prompt = "What is Artificial Intelligence? Explain in simple words."
models_to_try = [
    "gemini-3.5-flash",
    "gemini-2.5-flash-lite",
]
for model_name in models_to_try:
    try:
        print(f"Trying {model_name}...")
        response = client.models.generate_content(
            model=model_name,
            contents=prompt,
        )
        print("\nResponse:\n")
        print(response.text)
        break
    except errors.APIError as error:
        print(f"{model_name} failed: {error}")
else:
    print(
        "\nBoth models failed. Try again later, "
        "or check your Gemini API quota and service status."
```

```
    )
```

8. Execution

Run the Python program using the following command in the VS Code terminal:

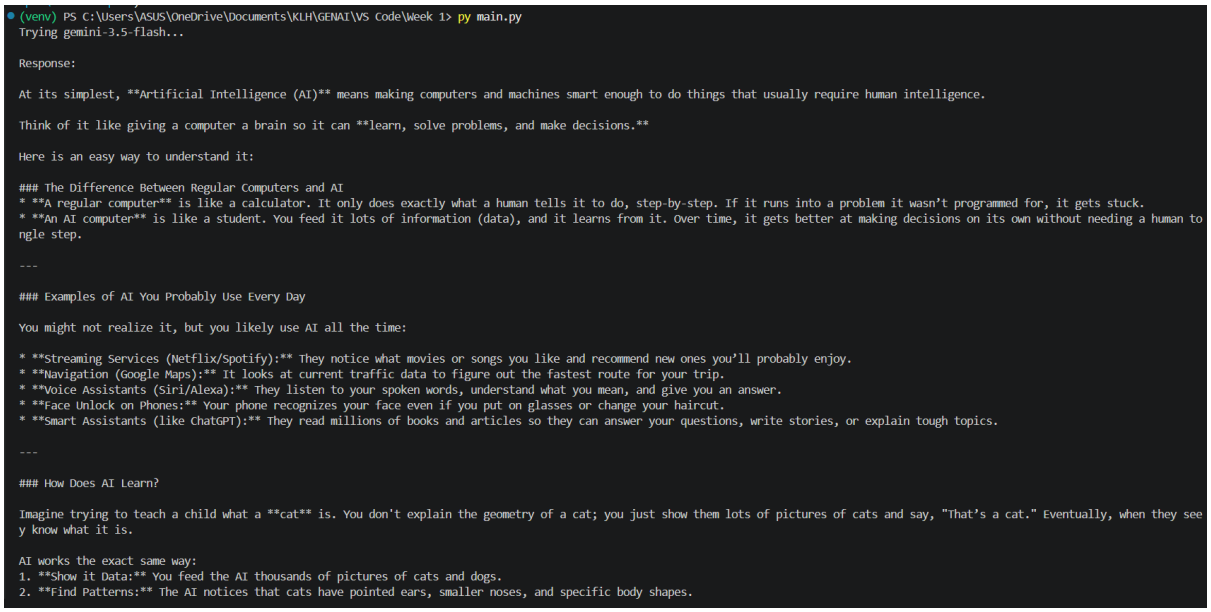
`python main.py`

The program sends the input prompt to the selected LLM model and displays the generated response in the terminal.

9. Output

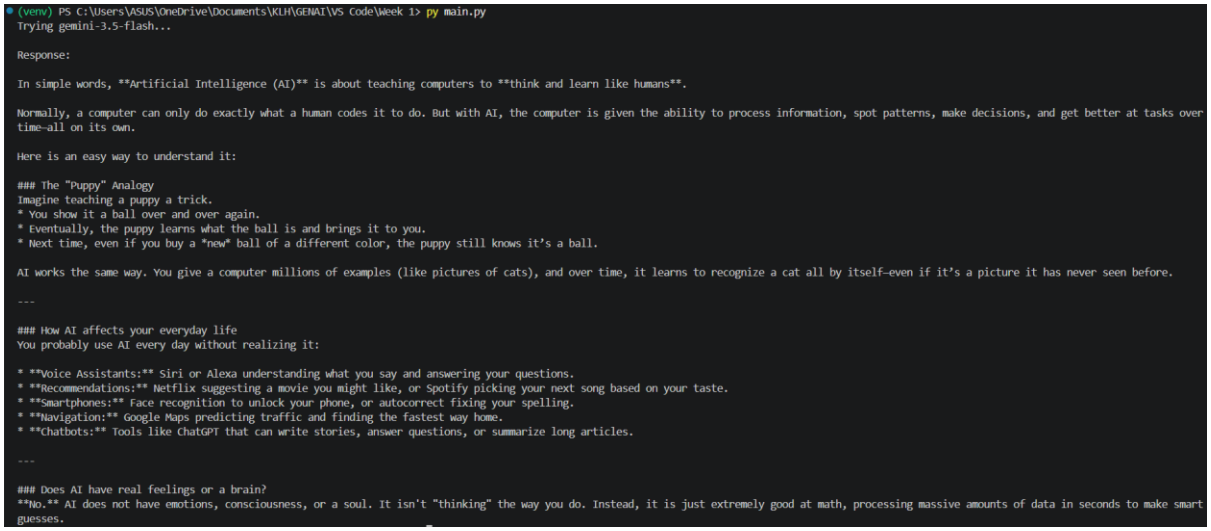
Output 1:

Prompt: What is Artificial Intelligence? Explain in simple words.



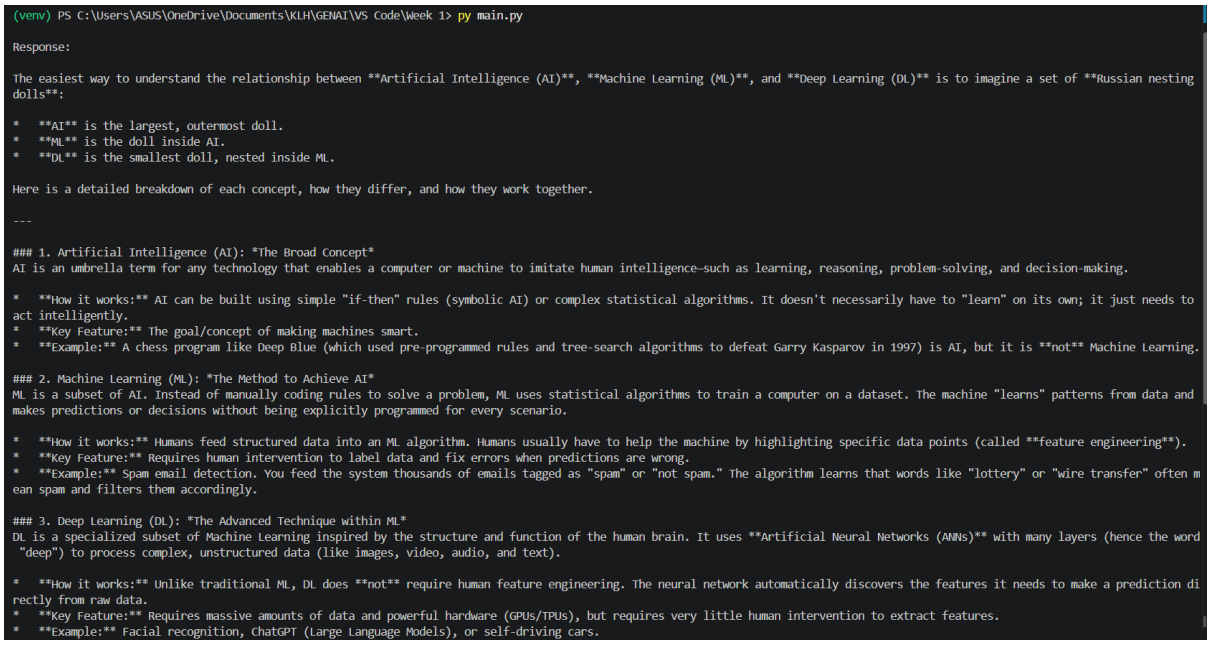
Output 2:

Prompt: Explain Generative AI with two real-world examples.



Output 3:

Prompt: Differentiate Artificial Intelligence, Machine Learning, and Deep Learning.



10. Observations

1. Successfully established a connection between Python and the OpenAI API.
2. Observed that the API key is required to authenticate requests.
3. The LLM generated responses based on the given prompts.
4. Different prompts produced different responses.
5. More specific prompts generated more focused responses.

11. Result

Successfully implemented the first LLM API call using Python and OpenAI. The application generated responses based on user-provided prompts.